

Beyond Trust: Building Community-Driven Security Analysis for Your .NET Software Supply Chain

Niels Tanis

Security Researcher & Engineer

TIDALIS



Who am I?



- Niels Tanis
- Security Researcher & Engineer
 - Background .NET Development, Pentesting/ethical hacking, and software security consultancy
 - Decade of research on static analysis for .NET apps
 - Principal Software Security Engineer at Tidalis
- Microsoft MVP – Developer Technologies



TIDALIS



@niels.fennec.dev

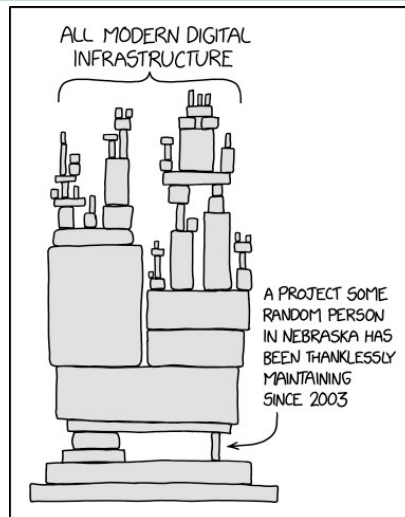


@nielstanis@infosec.exchange

Modern Application Architecture

XKCD 2347

#LUCK26



TIDALIS



@niels.fennec.dev



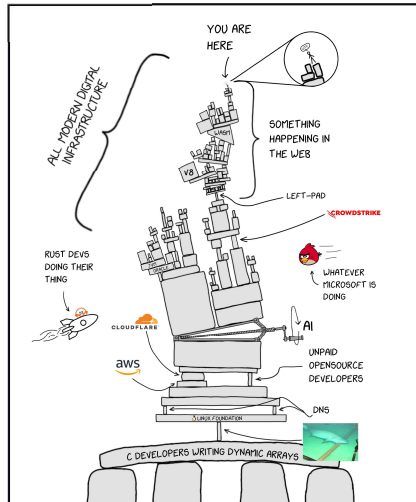
@nielstanis@infosec.exchange

<https://xkcd.com/2347/>

You are here



#LUC26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://xkcd.com/2347/>

Agenda



- Security challenges in our .NET software supply chain
- How can we start securing our supply-chain?
- Fennec Labs
 - Understanding what projects do for security
 - Know what's inside package
 - Review package
 - Future and idea's
- Conclusion and Q&A

TIDALIS



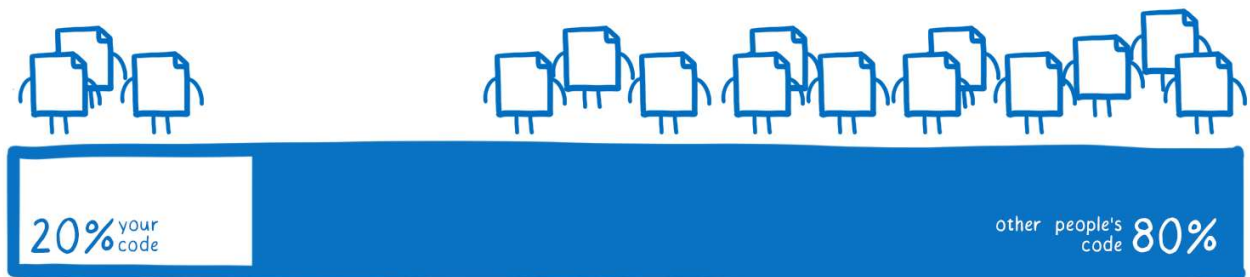
@niels.fennec.dev



@nielstanis@infosec.exchange

Average codebase composition

#HACK26



TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

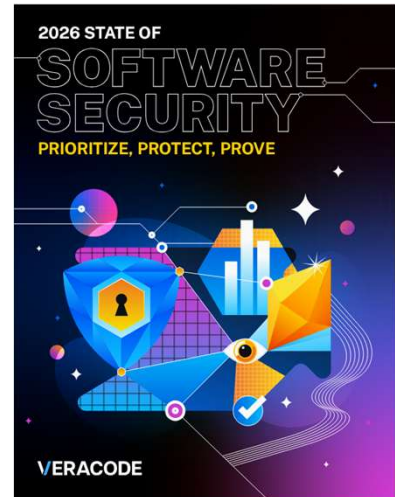
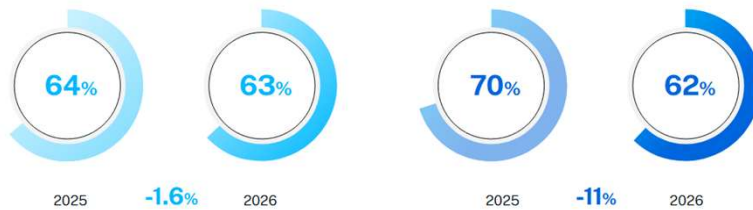
<https://hacks.mozilla.org/2019/11/announcing-the-bytecode-alliance/>

Veracode State Of Software Security 2026

#LUCK26

Prevalence of flaws in first-party (left) vs. third-party (right) code among applications

Percentage of flaws



TIDALIS



@niels.fennec.dev

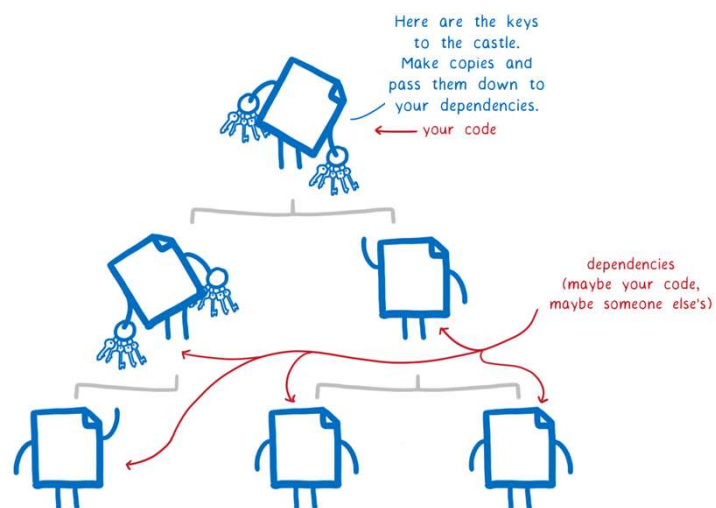


@nielstanis@infosec.exchange

<https://www.veracode.com/resources/analyst-reports/state-of-software-security-2026/>

Average codebase composition

#LUCCK26



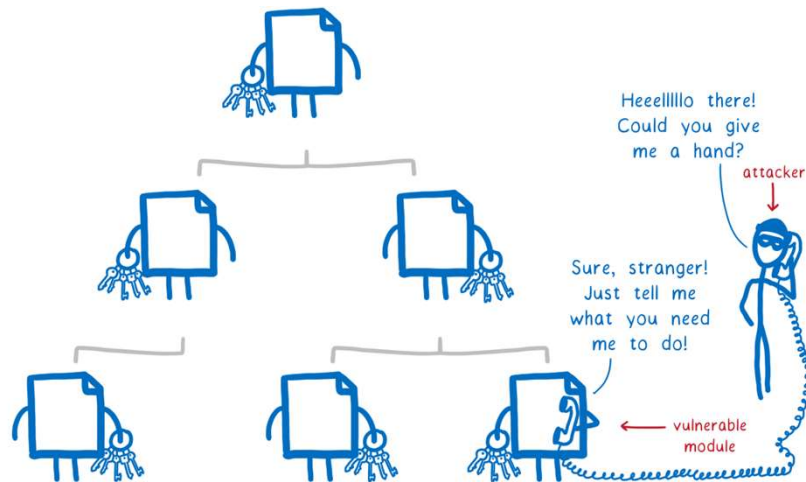
TIDALIS

@niels.fennec.dev @nielstanis@infosec.exchange

<https://hacks.mozilla.org/2019/11/announcing-the-bytecode-alliance/>

Vulnerable Assembly

#UCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://hacks.mozilla.org/2019/11/announcing-the-bytecode-alliance/>

DotNet CLI



```
nelson@ghost-m2:~/research/consoleapp
└─nelson at ghost-m2 in ~/research/consoleapp
   └─dotnet list package
      Project 'consoleapp' has the following package references
      [net9.0]:
      Top-level Package      Requested  Resolved
      > itext7               9.4.0     9.4.0

└─nelson at ghost-m2 in ~/research/consoleapp
   └─dotnet list package --vulnerable

The following sources were used:
  https://api.nuget.org/v3/index.json

The given project 'consoleapp' has no vulnerable packages given the current sources.
└─nelson at ghost-m2 in ~/research/consoleapp
```

TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

DotNet CLI



```
nelson@ghost-m2:~/research/consoleapp
└─nelson at ghost-m2 in ~/research/consoleapp
  └─dotnet list package --include-transitive
Project 'consoleapp' has the following package references
[net9.0]:
Top-level Package      Requested  Resolved
> itext7                9.4.0     9.4.0

Transitive Package      Resolved
> itext                  9.4.0
> itext.commons          9.4.0
> Microsoft.CSharp       4.0.1
> Microsoft.DotNet.PlatformAbstractions 1.1.0
> Microsoft.Extensions.DependencyInjection 5.0.0
> Microsoft.Extensions.DependencyInjection.Abstractions 5.0.0
> Microsoft.Extensions.DependencyModel    1.1.0
> Microsoft.Extensions.Logging            5.0.0
> Microsoft.Extensions.Logging.Abstractions 5.0.0
> Microsoft.Extensions.Options            5.0.0
> Microsoft.Extensions.Primitives         5.0.0
> Microsoft.NETCore.Platforms             1.1.1
> Microsoft.NETCore.Targets               1.1.3
> Microsoft.Win32.Primitives               4.3.0
> Microsoft.Win32.Registry                 4.3.0
> Newtonsoft.Json                         9.0.1
```

TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

DotNet CLI



```
nelson@ghost-m2:~/research/consoleapp
└─nelson at ghost-m2 in ~/research/consoleapp
  └─dotnet list package --include-transitive --vulnerable

The following sources were used:
  https://api.nuget.org/v3/index.json

Project 'consoleapp' has the following vulnerable packages
[net9.0]:
  Transitive Package      Resolved  Severity  Advisory URL
  > Newtonsoft.Json       9.0.1     High      https://github.com/advisories/GHSA-5crp-9r3c-p9vr

nelson at ghost-m2 in ~/research/consoleapp
└─
```

TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

DotNet CLI - .NET 10



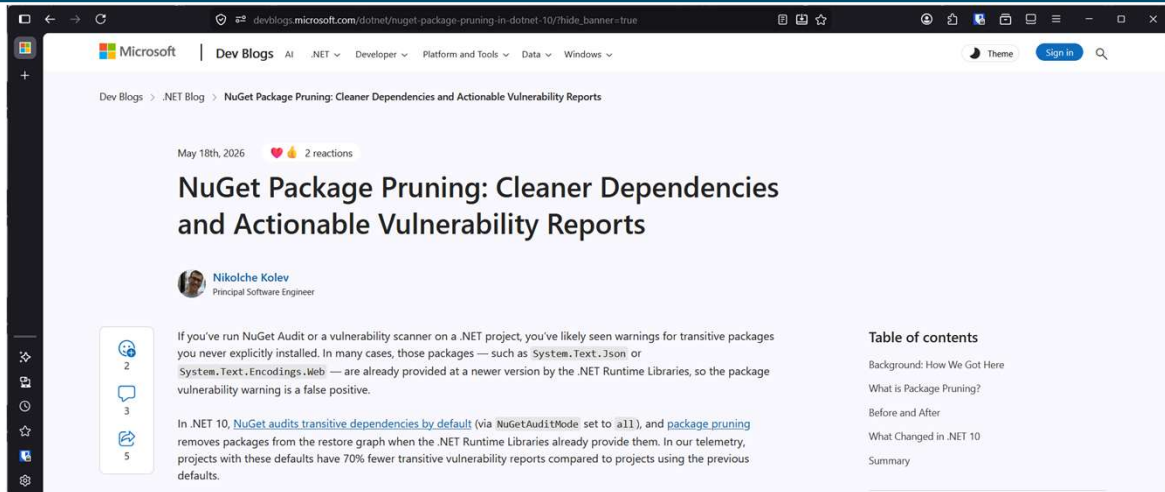
```
nelson@ghost-m2:~/research/consoleapp
└─nelson at ghost-m2 in ~/research/consoleapp
  └─dotnet --version
    10.0.100
└─nelson at ghost-m2 in ~/research/consoleapp
  └─dotnet build
    Restore succeeded with 1 warning(s) in 0.3s
    /Users/nelson/Research/consoleapp/consoleapp.csproj : warning NU1903: Package 'Newtonsoft.Json' 9.0.1 has a known high
    severity vulnerability, https://github.com/advisories/GHSA-5crp-9r3c-p9vr
    consoleapp net10.0 succeeded with 1 warning(s) (1.5s) → bin/Debug/net10.0/consoleapp.dll
    /Users/nelson/Research/consoleapp/consoleapp.csproj : warning NU1903: Package 'Newtonsoft.Json' 9.0.1 has a known high
    severity vulnerability, https://github.com/advisories/GHSA-5crp-9r3c-p9vr
    Build succeeded with 2 warning(s) in 2.0s
└─nelson at ghost-m2 in ~/research/consoleapp
  └─
```

TIDALIS

@niels.fennec.dev @nielstanis@infosec.exchange

NuGet Package Pruning

#LCK26



TIDALIS



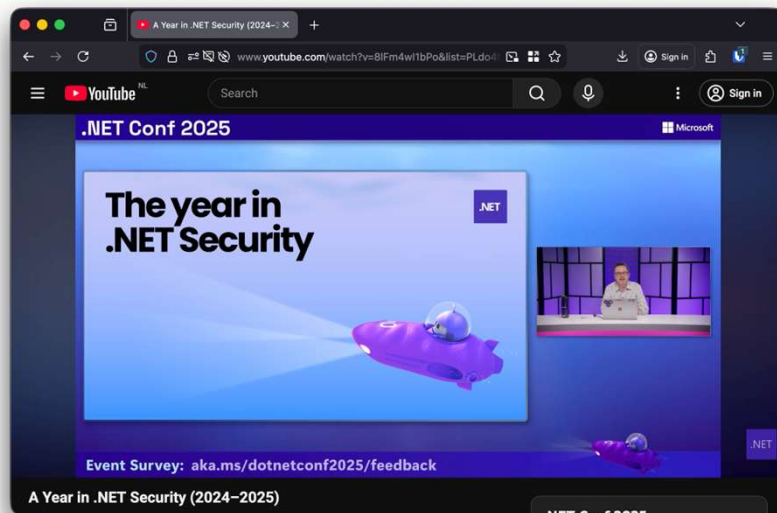
@niels.fennec.dev



@nielstanis@infosec.exchange

<https://devblogs.microsoft.com/dotnet/nuget-package-pruning-in-dotnet-10/>

.NET Conf 2025 – The year in .NET Security



TIDALIS

@niels.fennec.dev @nielstan's@infosec.exchange

https://youtu.be/8lFm4w1bPo?si=UJUwARMqY1NF_PO0

Summary of .NET Security Talk by Barry Dorrans

Barry Dorrans, Security PM for .NET, ASP.NET Core, and related technologies at Microsoft, presented an overview of security vulnerabilities in .NET over the past year.

Key Statistics

84 security reports submitted to Microsoft Security Response Center (MSRC)

12 CVEs issued (9 for .NET, 3 for ASP.NET Core)

\$93,000 paid out in bug bounties (ranging from \$3,000 to \$65,000)

Understanding Security Terminology

The talk explained important concepts:

CVE (Common Vulnerabilities and Exposures) - Standard vulnerability identification

CVSS (Common Vulnerability Scoring System) - Scores based on exploitability and impact, not necessarily risk

CWE (Common Weakness Enumeration) - Categories explaining why vulnerabilities exist

Notable Vulnerabilities by Type

4 Remote Code Executions (most serious)

2 Denial of Service

3 Elevation of Privilege

1 Spoofing

1 Security Feature Bypass

1 Information Disclosure

Key Takeaways for Developers

Avoid File.GetTempPath() - Use Directory.CreateTempSubdirectory() instead (available since .NET 7) to prevent Linux temp directory vulnerabilities

Validate all input - Never trust content from HTTP headers like Content-Disposition

Keep dependencies updated - But consider waiting a few days to avoid supply chain attacks

Minimize unsafe code - Buffer overflows remain a serious risk

Follow RFC specifications carefully - Pay attention to "must" vs "should" clauses

Bug Bounty Program

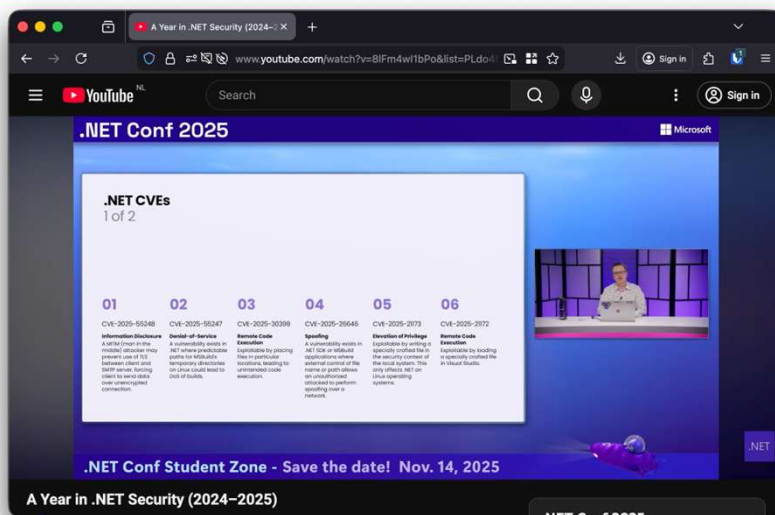
Microsoft pays for security vulnerabilities reported through MSRC (not GitHub issues), including vulnerabilities in documentation. Developers are encouraged to report issues to potentially receive bounty payments.

Important Reminder

.NET 8 and .NET 9 will only receive security patches for one more year after .NET 10's release, so upgrade planning should begin now.

.NET Conf 2025 – The year in .NET Security

#LUCK26



TIDALIS



@niels.fennec.dev



@nielstan's@infosec.exchange

https://youtu.be/8lFm4w1bPo?si=UJUwARMqY1NF_PO0

#UCK26

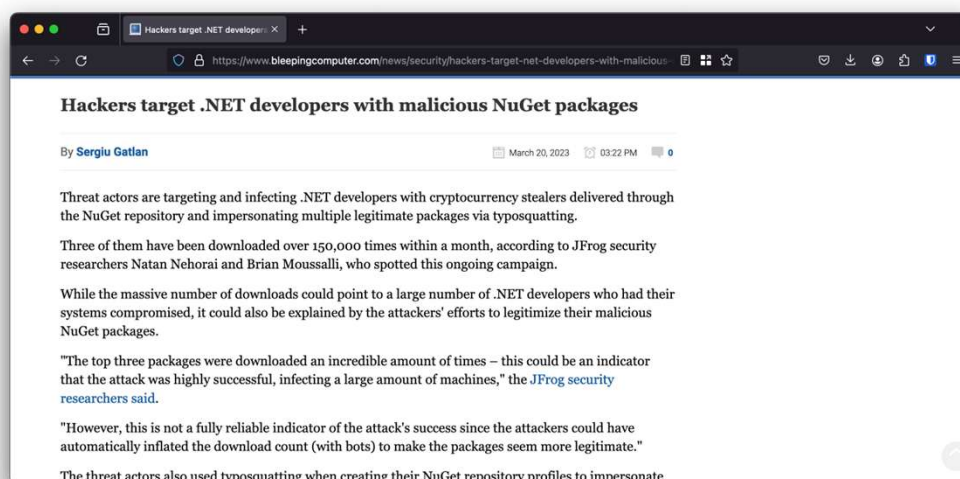


 @niels.fennec.dev @nielstanis@infosec.exchange

17

Malicious Package

#LUCK26



TIDALIS

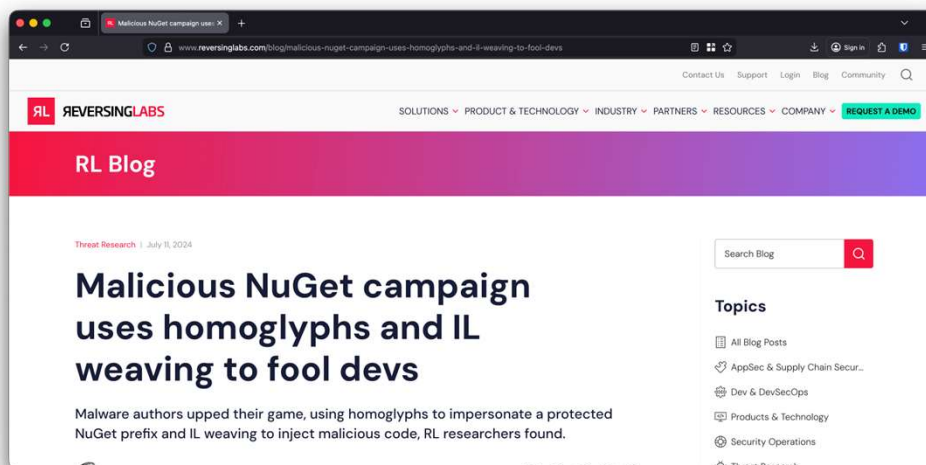
@niels.fennec.dev @nielstanis@infosec.exchange

<https://www.bleepingcomputer.com/news/security/hackers-target-net-developers-with-malicious-nuget-packages/>

Hackers targeting .NET developers with malicious NuGet packages:

- Threat actors delivered cryptocurrency stealers through typosquatted NuGet packages impersonating legitimate libraries
- Three malicious packages accumulated over 150,000 downloads within one month
- Attackers used typosquatting to create fake developer profiles mimicking Microsoft developers
- Malicious packages executed PowerShell dropper scripts (init.ps1) upon installation that set unrestricted execution policies
- Second-stage payload was a custom Windows executable (not typical commodity malware)
- Malware capabilities included stealing crypto wallets via Discord webhooks, extracting code from Electron archives, and auto-updating from C2 servers
- Some packages had no direct payload but fetched malicious code through booby-trapped dependencies
- Detection rates were very low - Microsoft Defender didn't flag these as malicious
- Developers should verify package names carefully, check for typos, and inspect installation scripts before installing packages

Malicious Package



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://www.reversinglabs.com/blog/malicious-nuget-campaign-uses-homoglyphs-and-il-weaving-to-fool-devs>

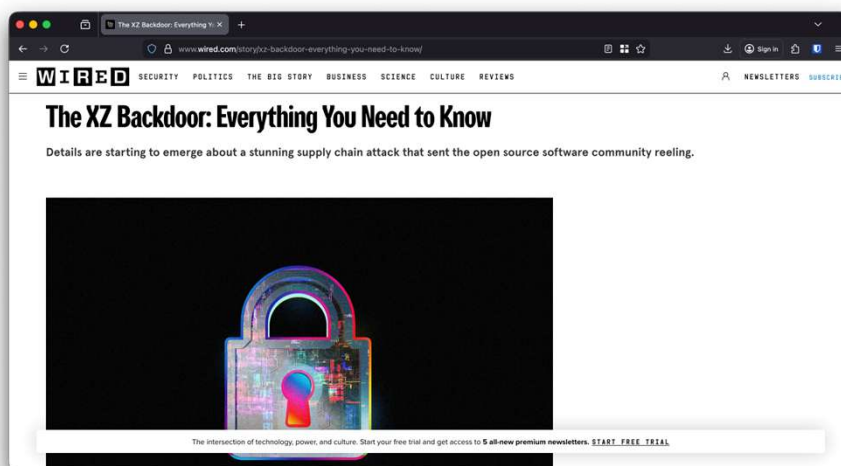
Key Attack Techniques

IL Weaving: Attackers used Intermediary Language weaving to patch legitimate .NET binaries and inject malicious module initializers [reversinglabs](https://www.reversinglabs.com). This made detection harder compared to earlier PowerShell-based approaches because the malicious code was embedded in compiled DLL files rather than plaintext scripts.

Homoglyphs for Typosquatting: The threat actors exploited homoglyphs—characters with identical visual appearance but different logical representations—to bypass NuGet's reserved prefix security feature [reversinglabs](https://www.reversinglabs.com). For example, they created a malicious package named "Guna.UI3.WinForms" (using Armenian and Cyrillic characters) to impersonate the legitimate "Guna.UI2.WinForms" package, which has a protected prefix.

Do you know what's inside? – xz utils

#L0CK26



TIDALIS



@niels.fennec.dev

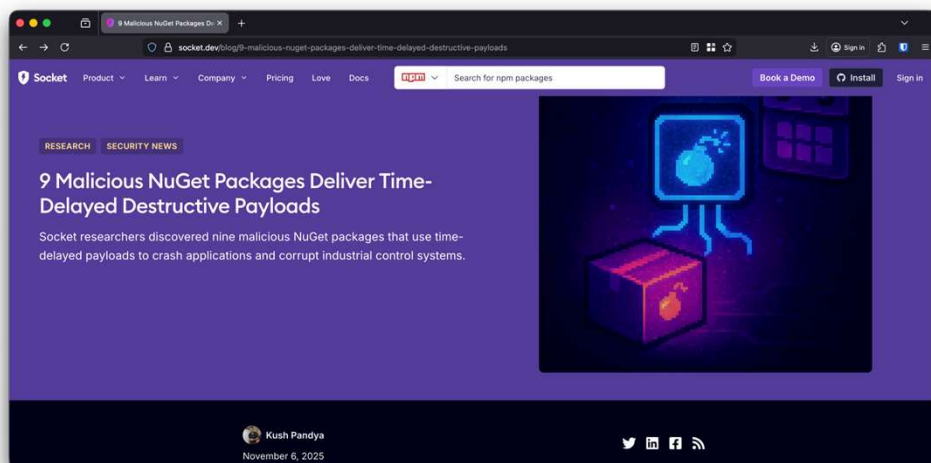


@nielstanis@infosec.exchange

<https://www.wired.com/story/xz-backdoor-everything-you-need-to-know/>

<https://www.youtube.com/watch?v=aoag03mSuXQ&t=597s&pp=ygUMdmVydGFzaXVtIHh6>

Malicious Package



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://socket.dev/blog/9-malicious-nuget-packages-deliver-time-delayed-destructive-payloads>

Key Takeaway

This represents a sophisticated evolution in supply chain attacks combining social engineering (functional code), technical stealth (extension methods), and strategic patience (multi-year delays) to make attribution and forensic analysis nearly impossible.

Malicious Package



- Single publisher called **shanghai666**
- 99% code is valid & functional focusing on databases and industrial PLC's, malicious extension methods
- Delayed logic for Aug '27 & Nov '28
- 20% of database queries are terminated
- Sharp7Extend (typo squatting) for PLC, immediate activation mimics hardware failures

TIDALIS



@niels.fennec.dev

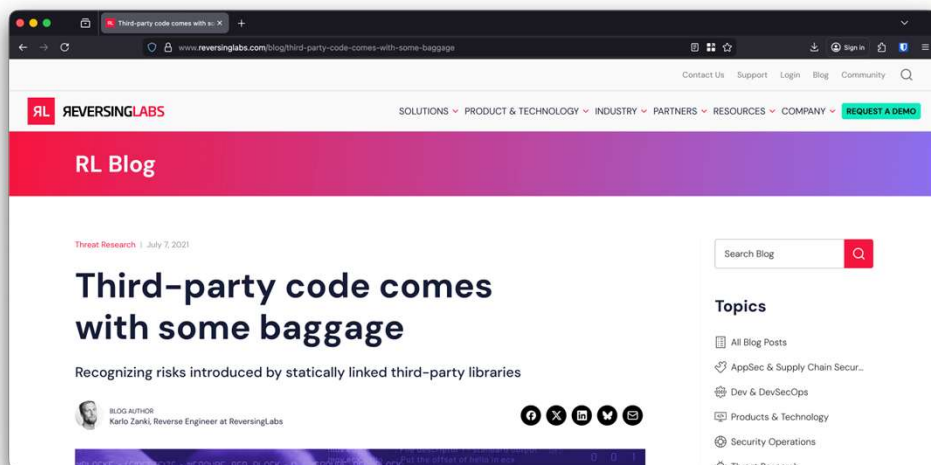


@nielstanis@infosec.exchange

<https://socket.dev/blog/9-malicious-nuget-packages-deliver-time-delayed-destructive-payloads>

Do you know what's inside?

#LUCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://www.reversinglabs.com/blog/third-party-code-comes-with-some-baggage>

Do you know what's inside?



- Unmanaged components interop
- Precompiled version of 7Zip, WinSCP and PuTTYgen
- Zlib compression library
 - Only source code distributed
 - Used in medical appliances
 - Severe vulnerabilities since 2005!

TIDALIS



@niels.fennec.dev

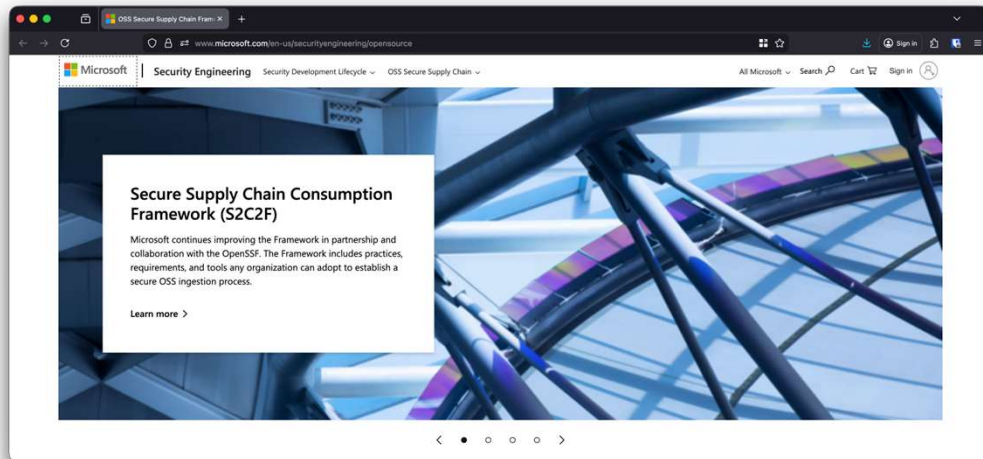


@nielstanis@infosec.exchange

<https://www.reversinglabs.com/blog/third-party-code-comes-with-some-baggage>

Secure Supply Chain Consumption Framework (S2C2F)

#LUCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Secure Supply Chain Consumption Framework (S2C2F)



Level 1	Level 2	Level 3	Level 4
Minimum OSS Governance Program • Use package managers [ING-1] • Local copy of artifact [ING-2] • Scan with known vulns [SCA-1] • Scan for software licenses [SCA-2] • Inventory OSS [INV-1] • Manual OSS updates [UPD-1]	Secure Consumption and Improved MTTR • Scan for end of life [SCA-3] • Have an incident response plan [INV-2] • Auto OSS updates [UPD-2] • Alerts on vulns at PR time [UPD-3] • Audit that consumption is through approved ingestion method [AUD-2] • Validate integrity of OSS [AUD-3] • Secure package source file configuration [ENF-1]	Malware Defense and Zero-Day Detection • Deny list capability [ING-3] • Clone OSS source [ING-4] • Scan for malware [SCA-4] • Proactive security reviews [SCA-5] • Enforce OSS provenance [AUD-1] • Enforce consumption from curated feed [ENF-2]	Advanced Threat Defense • Validate the SBOMs of OSS consumed [AUD-4] • Rebuild OSS on trusted infrastructure [REB-1] • Digitally sign rebuilt OSS [REB-2] • Generate SBOM for rebuilt OSS [REB-3] • Digitally sign protected SBOMs [REB-4] • Implement fixes [FIX-1]

TIDALIS



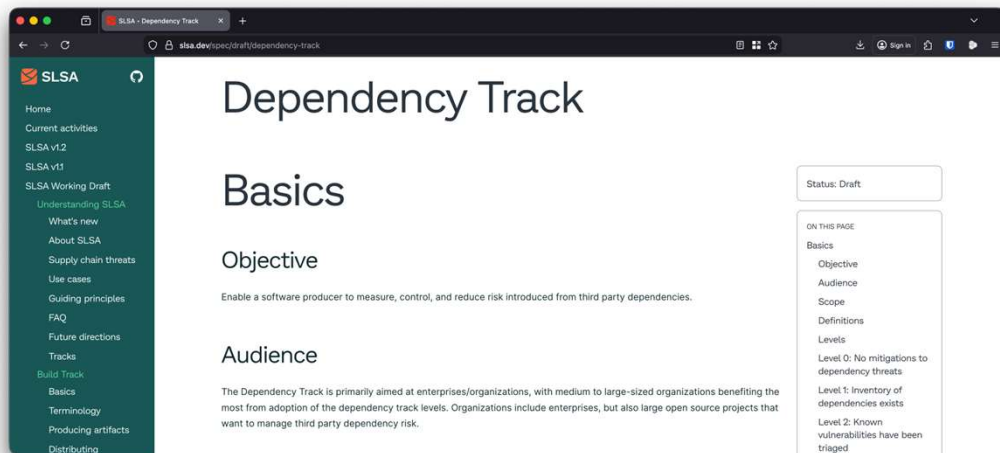
@niels.fennec.dev



@nielstanis@infosec.exchange

<https://github.com/ossf/s2c2f/blob/main/specification/framework.md>

S2C2F → SLSA Dependency Track



TIDALIS



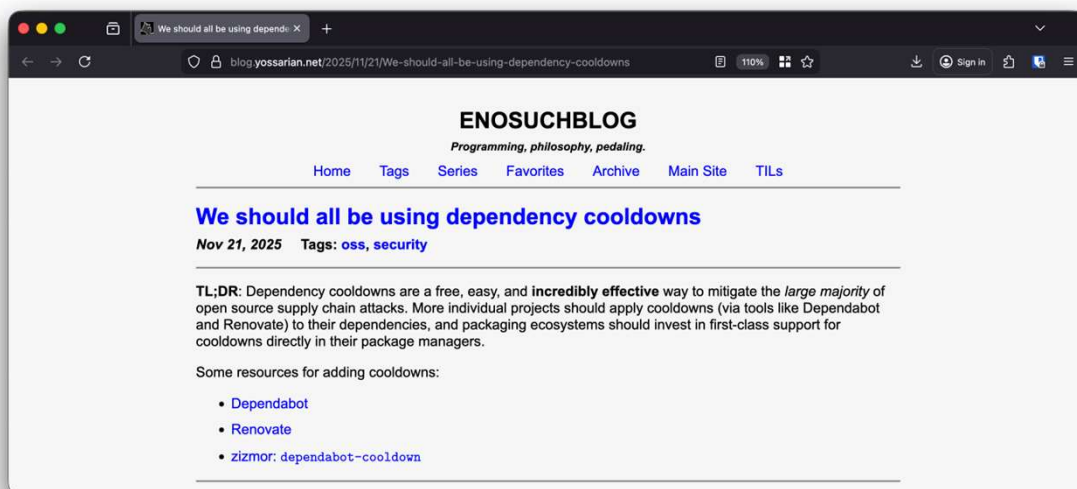
@niels.fennec.dev



@nielstanis@infosec.exchange

<https://slsa.dev/spec/draft/dependency-track>

Hot take; dependency cooldowns



TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

<https://blog.yossarian.net/2025/11/21/We-should-all-be-using-dependency-cooldowns>

What's inside? (Audit)

#LUCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://scorecard.dev/viewer/?uri=github.com%2FJamesNK%2FNewtonsoft.Json>

Open SSF Security Scorecard

#UCK26

- Set Automated Security Checks
- Scoring System 0-10
- Security Best Practices
- Ease of Use
- Community Support



TIDALIS

 @niels.fennec.dev

 @nielstanis@infosec.exchange

- Vulnerabilities (**High**)
 - Does the project have unfixed vulnerabilities?
- Maintained (**High**)
 - Is the project at least 90 days old, and maintained?
- Security Policy (**Medium**)
 - Does project contain security policy?

- License (**Low**)
 - Does the project declare a licence?
- CII Best Practices (**Low**)
 - Does the project have a CII Best Practices Badge?
- Dependency Update Tool (**High**)
 - Does the project use tools to help update its dependencies e.g. Dependabot, RenovateBot?

Continuous Testing



- Continuous Integration Tests (**Low**)
 - Does the project run tests in CI?
- Fuzzing (**Medium**)
 - Does the project use fuzzing tools, e.g. OSS-Fuzz?
- Static Analysis (**Medium**)
 - Does the project use static code analysis tools?

TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Source Risk Assessment



- Binary Artifacts (**High**)
 - Does repository contain binaries/compiled artifacts?
- Branch Protection (**High**)
 - GitHub best practices branch protection
- Dangerous Workflow (**Critical**)
 - GitHub best practices for Workflow Actions

TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

Source Risk Assessment



- Code Review (**High**)
 - Does the project require code review before code is merged?
- Contributors (**Low**)
 - Does the project have contributors from multiple organizations?

TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

Build Risk Assessment



- Pinned Dependencies (**Medium**)
 - Does the project declare and pin dependencies?
- Token Permission (**High**)
 - Does the project declare GitHub workflow tokens as read only?

TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Build Risk Assessment



- Packaging (**Medium**)
 - Does the project build and publish official packages from CI/CD?
- Signed Releases (**High**)
 - Does the project cryptographically sign releases?

TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Demo OpenSSF Scorecard



Running checks

TIDALIS

 [@niels.fennec.dev](#)  [@nielstanis@infosec.exchange](#)

Fennec Labs Scorecard

#LUCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://scorecard.dev/viewer/?uri=github.com%2FJamesNK%2FNewtonsoft.Json>

Fennec Labs Instrument

#UCK26

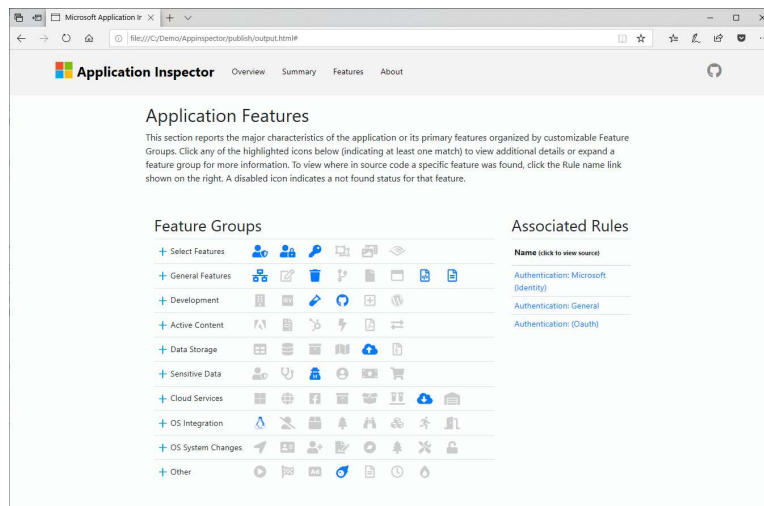


TIDALIS

[@niels.fennec.dev](#) [@nielstanis@infosec.exchange](#)

Application Inspector

#BUCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://github.com/microsoft/ApplicationInspector>

Application Inspector

#HACK26

Select Features

Feature	Confidence	Details
 Authentication		View
 Authorization		View
 Cryptography		View
 Object Deserialization		N/A
 AV Media Parsing		N/A
 Dynamic Command Execution		N/A

TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://github.com/microsoft/ApplicationInspector>

.NET Reproducibility



- Reproducible builds
 - Independent path from source to binary code.
- Roslyn Compiler Deterministic Compilation
- How reproducible is a simple console app?
 - Demo script available in repository

TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Fennec Labs Reproduce

#LUCK26



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Fennec Labs Compare

#UCK26



TIDALIS

 @niels.fennec.dev  @nielstanis@infosec.exchange

<https://scorecard.dev/viewer/?uri=github.com%2FJamesNK%2FNewtonsoft.Json>

Future Idea - Fuzzing

#LUCK26

- Fuzzing, or fuzz testing
 - Automated software testing method that uses a wide range of **invalid** and unexpected data as input to find flaws
- Good in C/C++ memory issues
- Can it be of any value with managed languages like .NET?



TIDALIS



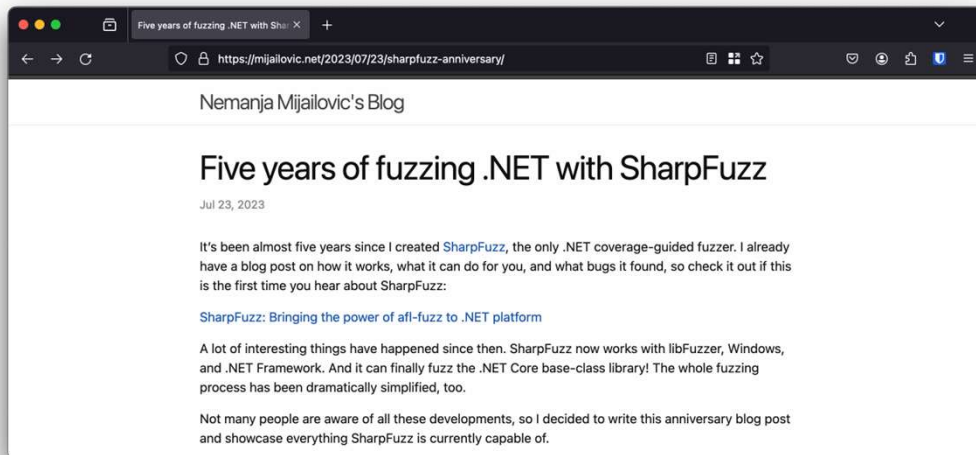
@niels.fennec.dev



@nielstanis@infosec.exchange

<https://www.bestpractices.dev/en/criteria/0>

Fuzzing .NET & SharpFuzz



TIDALIS



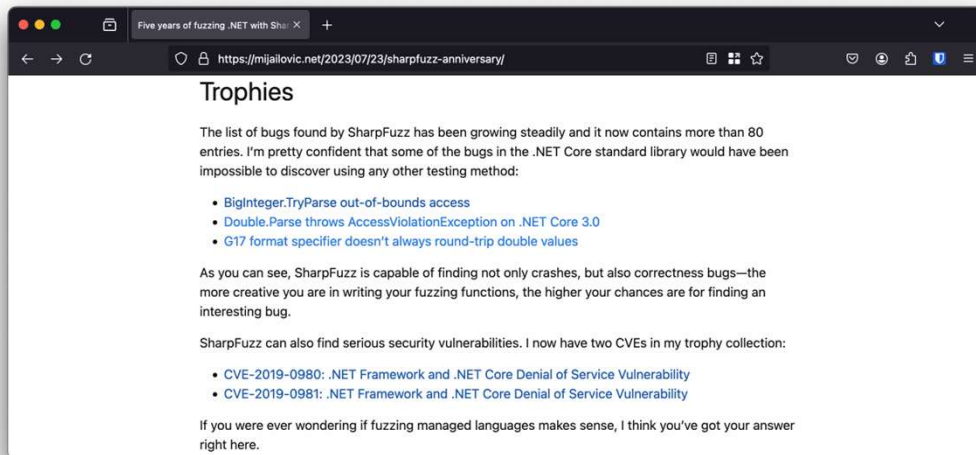
@niels.fennec.dev



@nielstanis@infosec.exchange

<https://mijailovic.net/2023/07/23/sharpfuzz-anniversary/>

Fuzzing .NET & SharpFuzz



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://mijailovic.net/2023/07/23/sharpfuzz-anniversary/>

Fuzzing .NET – Jil JSON Serializer



```
public static void Main(string[] args)
{
    SharpFuzz.Fuzzer.OutOfProcess.Run(stream => {
        try
        {
            using (var reader = new
                System.IO.StreamReader(stream))
                JSON.DeserializeDynamic(reader);
        }
        catch (DeserializationException) { }
    });
}
```

TIDALIS



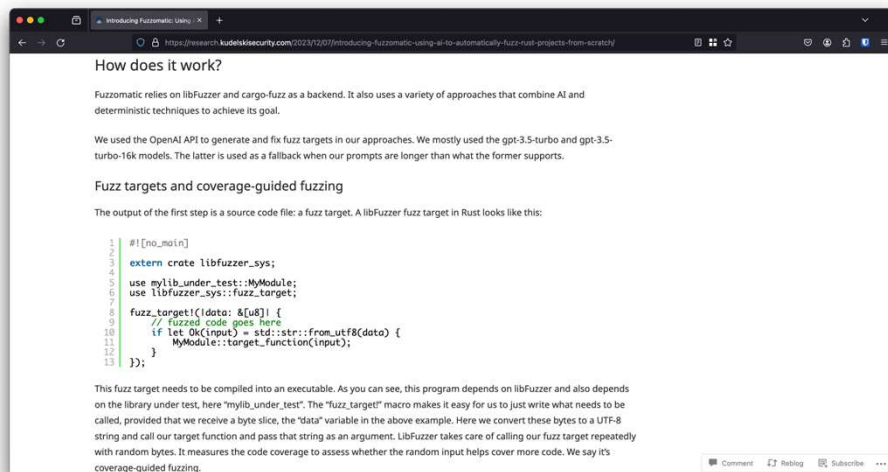
@niels.fennec.dev



@nielstanis@infosec.exchange

<https://github.com/google/fuzzing/blob/master/docs/structure-aware-fuzzing.md>

Fuzzomatic: Using AI to Fuzz Rust



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

<https://research.kudelskisecurity.com/2023/12/07/introducing-fuzzomatic-using-ai-to-automatically-fuzz-rust-projects-from-scratch/>

Future Idea – Dashboard & Feedback

#LUCK26

- Local and remote dashboard?
- Ability to do review and share security related data of project
- MCP server with embedded prompt resources to help security review
- Roslyn analyzer that can help guide the developer



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange

Conclusion



- Doing software security is hard and Software Security Professionals don't scale!
- Package eco-systems have been wild west lately
- Goal is to have Fennec Labs help with reviewing and sharing security details for all audiences
- Allowing everyone to make better informed decisions and can act if needed.

TIDALIS

 [@niels.fennec.dev](https://twitter.com/niels.fennec.dev)  @nielstanis@infosec.exchange

Thank you!

#LUC26

- niels at fennec.dev
- <https://github.com/nielstanis/updateconfkrakow2026>
- Questions?



TIDALIS



@niels.fennec.dev



@nielstanis@infosec.exchange